

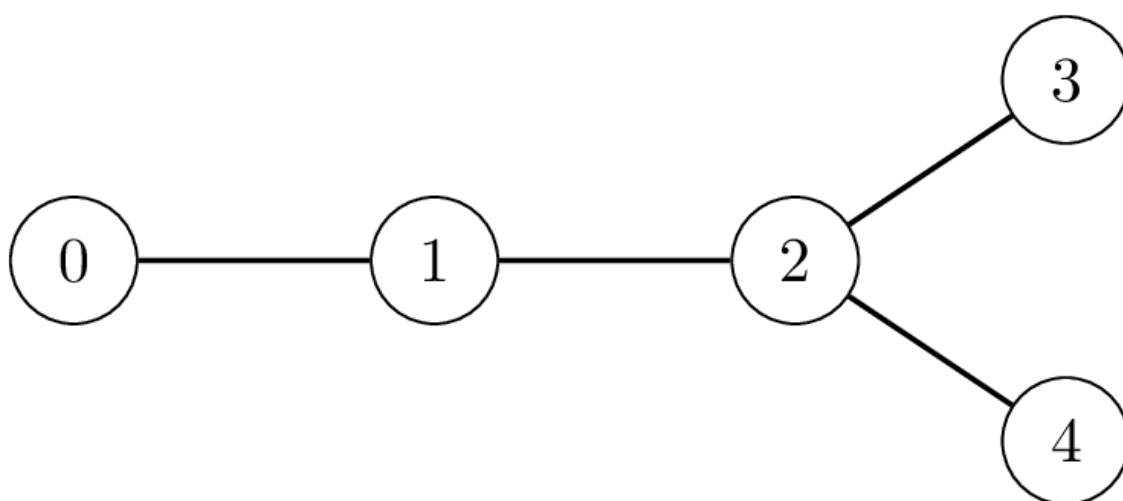
Migrations

Muzeul de Istorie Naturală investighează tiparele de migrare ale dinozaurilor din Bolivia. Paleontologii au descoperit urme de dinozauri în N situri diferite, numerotate de la 0 la $N - 1$ în ordine descrescătoare după vechime: situl 0 conține cele mai vechi urme, în timp ce situl $N - 1$ conține cele mai noi.

Dinozaurii au migrat la fiecare sit (cu excepția sitului 0) de la un alt sit mai vechi. Pentru fiecare sit i astfel încât $1 \leq i \leq N - 1$, există exact un sit mai vechi $P[i]$ (cu $P[i] < i$) astfel încât dinozaurii au migrat direct din situl $P[i]$ în situl i .

Paleontologii modelează fiecare migrație ca o **legătură nedirecționată** între siturile i și $P[i]$. Atenție la faptul că între oricare două situri distincte x și y , este posibil să putem călători de la x la y traversând o secvență de legături. **Distanța** între siturile x și y este definită ca fiind numărul minim de legături necesare ca să călătorești de la x la y .

Spre exemplu, următoarea imagine ilustrează cazul cu $N = 5$ situri, unde $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, and $P[4] = 2$. Putem călători de la situl 3 la situl 4 prin 2 conexiuni, așa că distanța dintre cele două este 2.



Muzeul intenționează să identifice perechi de situri cu distanța maximă posibilă între ele. Observați că perechea nu este neapărat unică: de pildă, în exemplul de mai sus, ambele perechi $(0, 3)$ și

$(0, 4)$ au distanța 3, care este maximă. În astfel de cazuri, orice pereche de distanță maximă este considerată corectă.

Inițial, valorile $P[i]$ **nu** sunt cunoscute. Muzeul trimite o echipă de cercetători să viziteze siturile $1, 2, \dots, N - 1$, în ordine. Ajunși la situl i ($1 \leq i \leq N - 1$), echipa execută ambele dintre următoarele două acțiuni:

- Determină valoarea $P[i]$, aceasta fiind sursa de migrație la situl i .
- Decid dacă trimite **un** mesaj la Muzeu sau să nu trimită niciun mesaj de la acest sit, bazat pe informațiile adunate până acum.

Mesajele sunt transmise prin intermediul unui sistem costisitor de comunicare prin satelit, așa că fiecare mesaj trebuie să fie un număr întreg între 1 și 20 000, inclusiv. În plus, echipa are dreptul să trimită **cel mult 50 de mesaje** în total.

Sarcina ta este să implementezi o strategie prin care:

- Echipa de cercetători selectează siturile din care se vor trimite mesaje, împreună cu valorile fiecărui mesaj.
- Muzeul poate determina o pereche de situri cu distanța maximă, bazându-se exclusiv pe mesajele primite de la fiecare sit, știind de la ce situri au fost trimise aceste mesaje.

Este costisitor să trimiți mesaje mari prin satelit. Scorul tău va depinde atât de valoarea maximă trimisă cât și de numărul total de mesaje trimise.

Implementation Details

Trebuie să implementezi două funcții: una pentru echipa de cercetare, iar cealaltă pentru Muzeu.

Funcția pe care trebuie să o implementezi pentru **echipa de cercetare** este:

```
int send_message(int N, int i, int Pi)
```

- N : numărul de situri.
- i : indexul sitului pe care echipa îl vizitează la momentul actual.
- Pi : valoarea lui $P[i]$.
- Funcția este apelată de $N - 1$ ori pentru fiecare test, în ordinea $i = 1, 2, \dots, N - 1$.

Această funcție trebuie să returneze $S[i]$ specificând acțiunea pe care echipa o îndeplinește la situl i :

- $S[i] = 0$: echipa de cercetare decide să nu trimită mesaj de la situl i .
- $1 \leq S[i] \leq 20\,000$: echipa de cercetare trimite mesajul $S[i]$ de la situl i .

Funcția pe care trebuie să o implementezi pentru **Muzeu** este:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : un tablou de lungime N astfel încât:
 - $S[0] = 0$.
 - Pentru fiecare $1 \leq i \leq N - 1$, $S[i]$ este numărul întreg returnat de `send_message(N, i, P[i])`.
- Această funcție este apelată o singură dată în cadrul unui test.

Această funcție trebuie să returneze o pereche de situri (U, V) cu distanță maximă.

În evaluare, un program care apelează funcțiile de mai sus este rulat **exact de două ori**.

- La prima rulare:
 - `send_message` este apelată de $N - 1$ ori.
 - **Programul vostru poate stoca și reține informații la apeluri succesive.**
 - Valorile returnate (tabloul S) sunt stocate de sistemul de evaluare.
 - În anumite cazuri, comportamentul evaluatorului este **adaptiv**. Asta înseamnă că valorile $P[i]$ într-un apel `send_message` pot depinde de acțiunile precedente ale echipei de cercetare.
- La a doua rulare a programului:
 - `longest_path` este apelată o singură dată. Singura informație disponibilă funcției `longest_path` din prima rulare este tabloul S .

Restricții

- $N = 10\,000$
- $0 \leq P[i] < i$ pentru fiecare i astfel încât $1 \leq i \leq N - 1$.

Subtask-uri și Punctare

Subtask	Scor	Restricții suplimentare
1	30	Situl 0 și un alt sit formează o pereche de distanță maximă.
2	70	Fără restricții suplimentare.

Fie Z numărul maxim care apare în S , iar M numărul de mesaje trimise de echipa de cercetare.

În orice test, dacă cel puțin una dintre condițiile de mai jos este adevărată, atunci scorul soluției tale pe acel test este 0 (raportat ca și `Output isn't correct` în CMS):

- Cel puțin un element din S este invalid.
- $Z > 20\,000$ sau $M > 50$.
- Valoarea returnată de apelul funcției `longest_path` este incorectă.

Altfel, scorul tău pe subtask-ul 1 este calculat după cum urmează:

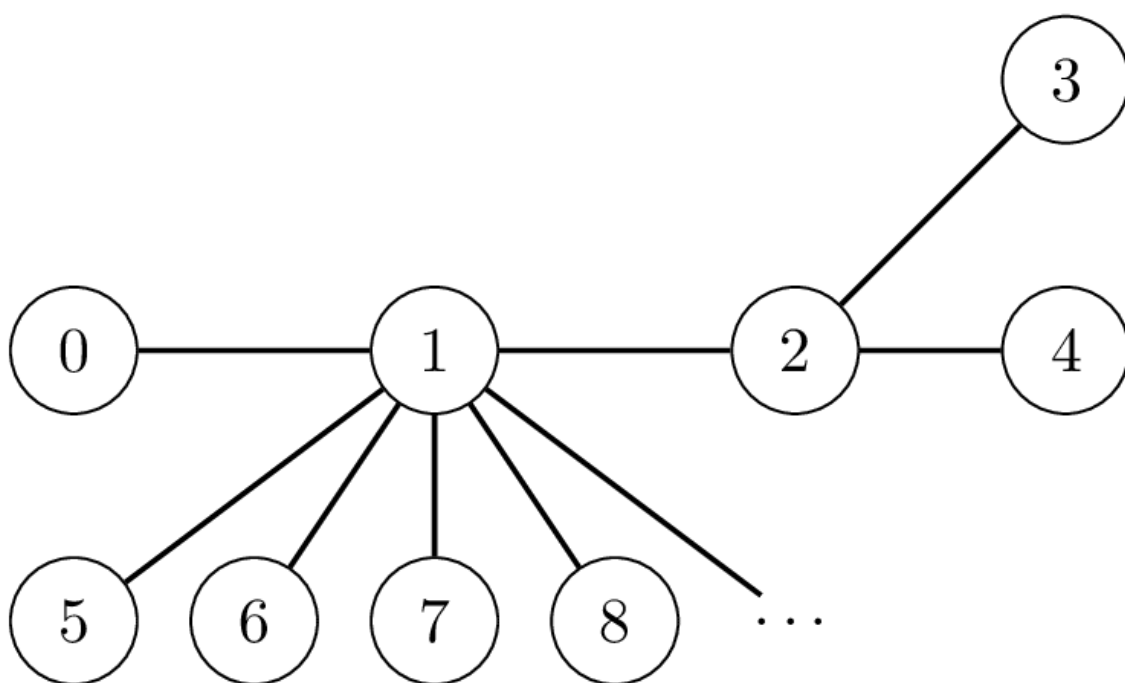
Condiție	Scor
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

Scorul tău pe subtask-ul 2 este calculat după cum urmează:

Condiție	Scor
$5 \leq Z \leq 20\,000$ și $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ and $32 \leq M \leq 50$	40
$Z \leq 4$ and $9 < M < 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ and $M \leq 8$	70

Exemplu

Fie $N = 10\,000$. Să considerăm următoarea situație în care $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$, iar $P[i] = 1$ for $i > 4$.



Să presupunem că strategia echipei de cercetare este să trimită mesajul $10 \cdot V + U$ de fiecare dată când perechea (U, V) cu distanță maximă se schimbă ca rezultat al apelului `send_message`.

Inițial perechea cu distanță maximă este $(U, V) = (0, 0)$. Să considerăm următoarea secvență de apeluri în cadrul primei rulări ale programului:

Apel funcție	(U, V)	Valoare returnată
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Observați că în toate celelalte apeluri valoarea lui $P[i]$ este 1. Asta înseamnă că perechea de distanță maximă nu se schimbă, iar echipa nu mai trimite mesaje.

Apoi, în a doua rulare a programului, se face următorul apel:

```
longest_path([0, 10, 20, 30, 0, ...])
```

Muzeul citește ultimul mesaj trimis de echipa de cercetare, care este $S[3] = 30$ și deduce că perechea $(0, 3)$ este cea cu distanță maximă. Astfel acest apel returnează $(0, 3)$.

Observați că această abordare nu poate mereu să ajute Muzeul să determine corect perechea cu distanță maximă.

Grader-ul local

Graderul local apelează atât `send_message` cât și `longest_path` în cadrul aceleiași rulări, ceea ce este diferit față de grader-ul real.

Format intrare:

```
N
P[1] P[2] ... P[N-1]
```

Format ieșire:

```
S[1] S[2] ... S[N-1]
U V
```

Observați că puteți folosi grader-ul cu orice valoare a lui N .